

교재명	2025 이패스 AICE Associate
작성일	2025.8.21

페이지	정오항목	수정 전	수정 후
50	코드	Numpy는 파이썬 패키지 관리 도구인 pip 명령어를 통해 쉽게 설치할 수 있습니다. <pre>코드 pip install numpy</pre>	install
		<pre>코드 pip install numpy --upgrade</pre>	install
67	코드동작 설명	<pre>코드 # HTML 데이터 읽어들이기 url_html = "https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)" tables = pd.read_html(url_html) # HTML 페이지의 모든 데이터를 읽어들이기 gdp_table = tables[2] # 해당 데이터를 선택 (해당 GDP 데이터) print(gdp_table.head())</pre> • HTML 테이블 형식이란? HTML은 웹 페이지를 구성하는 마크업 언어입니다. (table)태그를 사용해 데이터를 테이블 형식으로 표현할 수 있습니다. • 코드 동작 pd.read_html()은 지정된 URL에서 HTML 파일을 읽어 모든 테이블을 리스트 형태로 반환합니다. tables[2]는 반환된 리스트에서 3번째 테이블을 선택합니다. ○ 선택된 테이블을 DataFrame 형식으로 변환한 후, 첫 5행을 출력합니다.	<pre>print(gdp_table.head())</pre> 문구 추가
68	코드 & 문구 수정	① CSV 파일로 저장 <pre>DataFrame.to_csv() 메서드를 사용합니다.</pre> • 저장 경로와 파일 이름을 지정하고, 옵션으로 인덱스 포함 여부 등을 설정할 수 있습니다. <pre>코드 # data 변수는 CSV파일 불러오기 (70)에 (2) 참고해서 코드상 위와 동일 data.to_csv('processed_wine_data.csv', index=False)</pre>	<pre>DataFrame.to_csv() => data.to_csv</pre> 70 => 65
		<pre>코드 # data 변수는 CSV파일 불러오기 (70)에 (2) 참고해서 코드상 위와 동일 wine_data.to_excel('processed_wine_data.xlsx', index=False)</pre> • 주요 옵션 sheet_name : 저장할 시트의 이름을 지정합니다. 기본값은 'Sheet1'입니다. index : 행 인덱스를 포함할지 여부를 설정합니다. ③ JSON 파일로 저장 • DataFrame.to_json() 메서드를 사용합니다. • JSON 파일 형식은 계층적 데이터 구조를 표현하기 적합합니다. <pre>코드 # data 변수는 CSV파일 불러오기 (70)에 (2) 참고해서 코드상 위와 동일 data.to_json('processed_wine_data.json', orient='records')</pre>	70 => 65
166	코드 수정	⑤ 교차표 비율계산 : 전체 비율 <pre>코드 # 전체 기준 비율 계산 total_percentage = pd.crosstab(df['sex'], df['output'], normalize=True) * 100 print("\n전체 기준 비율:\n") total_percentage</pre>	) 삭제

257

코드 수정

(2) K-NN(K-Nearest Neighbors)분류: 심장병 예측 실험 (heart.csv 활용)
① 모델생성 코드

```
print('정확도(Accuracy)', accuracy_score(y_test, y_pred))
print('혼동 행렬\n', confusion_matrix(y_test, y_pred))

# 필요한 라이브러리 불러오기
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# 1. 데이터 로드
# 'heart.csv' 파일을 읽어와 데이터프레임 생성
data = pd.read_csv('datasets/heart.csv')

# 2. 입력값과 출력값 분리
# 'target' 값이 0이면 0, 1이면 1로 설정
X = data.drop('output', axis=1)
y = data['output']

# 3. 데이터를 학습용과 테스트용으로 나누기
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. 모델 생성 및 학습
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# 5. 테스트 데이터로 예측 수행
y_pred = model.predict(X_test)

# 6. 모델 평가
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

# 결과 출력
print('Accuracy:', accuracy)
print('Confusion Matrix\n', conf_matrix)
```

필요한 라이브러리 불러오기

```
import pandas as pd
from sklearn.model_selection import
train_test_split
from sklearn.neighbors import
KNeighborsClassifier
from sklearn.metrics import
accuracy_score, confusion_matrix
import warnings
warnings.filterwarnings('ignore')
```

1. 데이터 불러오기

df = pd.read_csv('datasets/heart.csv')

2. X, y 분리

```
X = df.drop('output', axis=1) # 입력
특성,
y = df['output'] # 예측 대상 (심장병
여부)
```

3. 데이터 분할

```
X_train, X_test, y_train, y_test =
train_test_split
X, y, test_size=0.2, random_state=42)
```

4. K-NN 모델 생성(K=5)

```
model =
KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
```

5. 예측 및 평가

```
y_pred = model.predict(X_test),
print('정확도(Accuracy):
accuracy_score(y_test, y_pred))
print('혼동 행렬: confusion_matrix(y_test,
y_pred))
```